

Module M214

Application Cloud Native

Architecture Microservices

Introduction et Concepts

Du Monolithe aux Microservices

OFPPT

Développement Digital - Option Full Stack

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs du chapitre	2
2	Architecture Monolithique	3
2.1	Définition	3
2.2	Structure d'une application monolithique	3
2.3	Exemple concret	3
2.4	Problèmes de l'architecture monolithique	4
2.5	Illustration du problème de scalabilité	4
3	Architecture Microservices	5
3.1	Définition	5
3.2	Principe fondamental	5
3.3	Comparaison visuelle	6
3.4	Exemple : Site e-commerce en Microservices	6
4	Concepts Clés des Microservices	7
4.1	Autonomie technologique	7
4.2	Déploiement ciblé	8
4.3	Scalabilité (Mise à l'échelle)	8
4.4	Inconvénients des Microservices	9
4.5	Qui utilise les Microservices ?	9
5	Communication entre Services	10
5.1	Les trois modes de communication	10
5.2	Communication Synchrone	10
5.3	Communication Asynchrone	10
5.4	Diffusion d'Événements	11
5.5	Comparaison des modes	11
6	API Gateway	12
6.1	Définition	12
6.2	Architecture avec API Gateway	12
6.3	Technologies	12
7	Étude de Cas : Gestion de Bibliothèque	13
7.1	Contexte	13
7.2	Identification des services	13
7.3	API du Service Livre	13
7.4	Architecture complète	14
8	Synthèse	15
8.1	Comparatif	15
8.2	Quand choisir quoi ?	15

1 Introduction

1.1 Contexte

Avec l'évolution des besoins des entreprises et l'augmentation du trafic sur les applications web, les architectures traditionnelles ont montré leurs limites. Les géants du web comme Netflix, Amazon, Uber et Spotify ont dû repenser la façon de concevoir leurs applications pour répondre à des millions d'utilisateurs simultanés.

Cette évolution a donné naissance à une nouvelle approche architecturale : les **Micro-services**.

1.2 Objectifs du chapitre

À la fin de ce chapitre, vous serez capable de :

- Comprendre la différence entre architecture monolithique et microservices
- Identifier les avantages et inconvénients de chaque approche
- Concevoir une architecture microservices simple
- Comprendre les modes de communication entre services
- Connaître le rôle d'une API Gateway

2 Architecture Monolithique

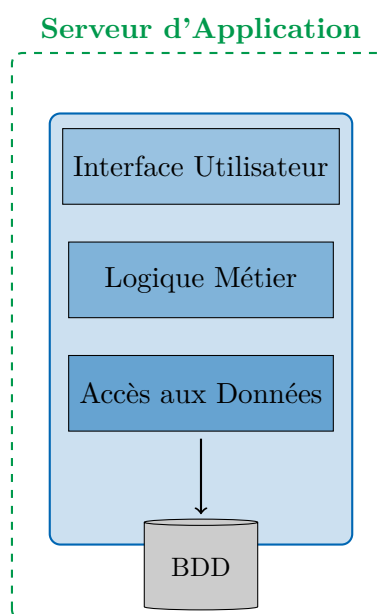
2.1 Définition

Architecture Monolithique

Une application **monolithique** est une application développée en un **seul bloc**, avec une même technologie, et déployée dans un seul serveur d'application.

Tous les composants de l'application (interface utilisateur, logique métier, accès aux données) sont regroupés dans une seule unité de déploiement (WAR, JAR, EAR, DLL, etc.).

2.2 Structure d'une application monolithique



2.3 Exemple concret

Prenons l'exemple d'un **site e-commerce** monolithique :

- **Module Client** : gestion des comptes utilisateurs
- **Module Produit** : catalogue des produits
- **Module Commande** : gestion des commandes
- **Module Paiement** : traitement des paiements

Dans une architecture monolithique, tous ces modules sont dans le **même code source**, la **même base de données**, et déployés **ensemble**.

2.4 Problèmes de l'architecture monolithique

✖ Difficultés du monolithe

1. Déploiement compliqué

- Tout changement nécessite le redéploiement de **toute l'application**
- Un bug dans un module peut faire tomber **l'ensemble du système**

2. Scalabilité non optimisée

- Pour augmenter les performances, il faut dupliquer **toute l'application**
- Impossible de scaler uniquement le module qui en a besoin

3. Autres problèmes

- Code de plus en plus complexe avec le temps
- Difficile de changer de technologie
- Équipes dépendantes les unes des autres

2.5 Illustration du problème de scalabilité

👤 Fort trafic sur le module Produit



On duplique TOUT l'application,
même les modules non sollicités!

Ces difficultés ont donné naissance à l'architecture **Microservices**.

3 Architecture Microservices

3.1 Définition

Architecture Microservices

L'architecture **Microservices** décompose une application en un ensemble de **petits services indépendants**, chacun :

- Exécutant une **fonction métier spécifique**
- Possédant sa **propre base de données**
- Pouvant être **déployé indépendamment**
- Communiquant via des **APIs** (généralement REST)
- Pouvant utiliser des **technologies différentes**

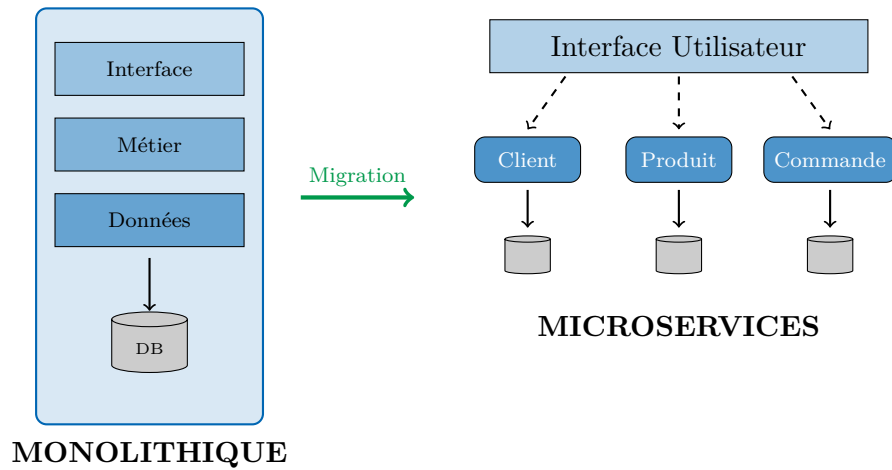
3.2 Principe fondamental

Principe des Microservices

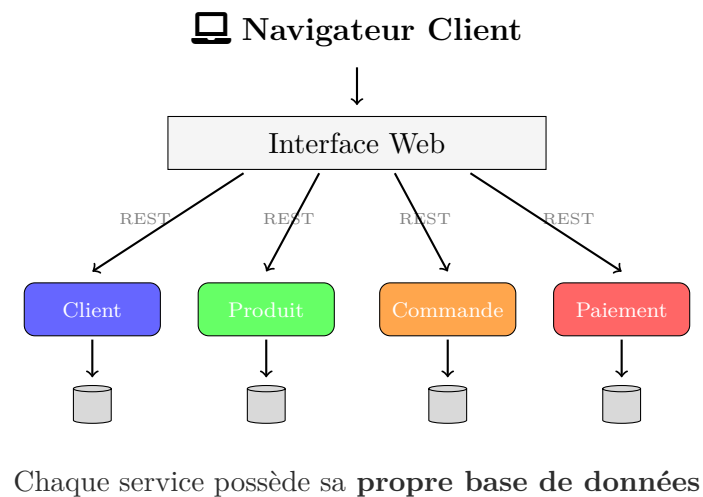
Découper une application en **petits services autonomes** qui exposent une **API** que les autres microservices peuvent consommer.

Chaque service est responsable d'une seule fonctionnalité et peut être développé, déployé et mis à l'échelle **indépendamment**.

3.3 Comparaison visuelle



3.4 Exemple : Site e-commerce en Microservices



✓ Avantage clé

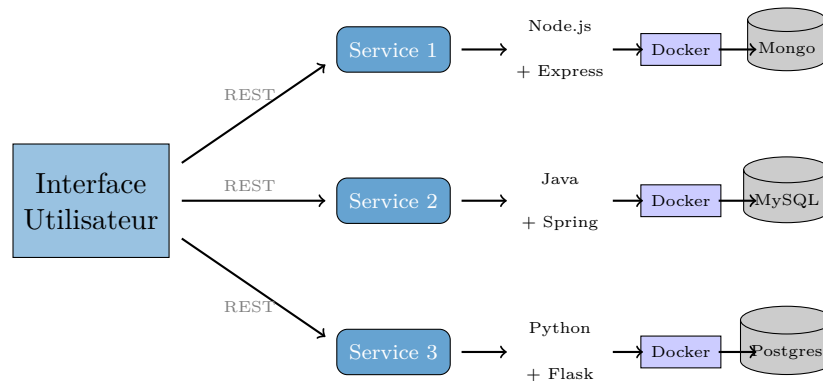
Un dysfonctionnement du service **Païement** n'arrêtera pas les autres services. Le système reste partiellement opérationnel.

4 Concepts Clés des Microservices

4.1 Autonomie technologique

Chaque microservice est **parfaitement autonome** :

- Sa propre **base de données** (MySQL, MongoDB, PostgreSQL...)
- Son propre **serveur d'application** (Apache, Tomcat, Node.js...)
- Ses propres **librairies et frameworks**
- Son propre **langage de programmation**



Conteneurisation avec Docker

Les microservices sont généralement déployés dans des **conteneurs Docker**, ce qui les rend portables, isolés et faciles à déployer.

4.2 Déploiement ciblé

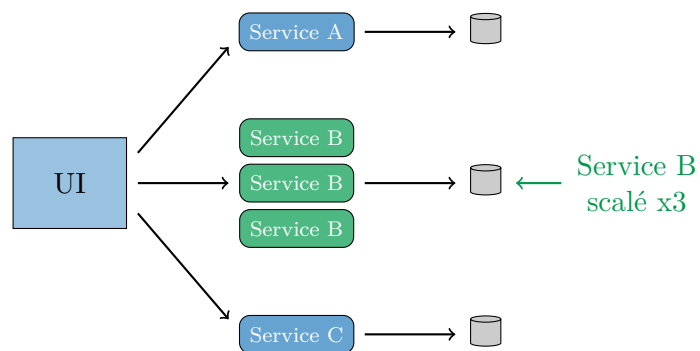
Chaque service peut avoir sa propre version et être déployé indépendamment :

Service Client	v2.0	✓ Déploiement indépendant
Service Produit	v10.2	✓ Moins de coordination
Service Commande	v5.3	✓ Plus rapide
		✓ Moins de risques

4.3 Scalabilité (Mise à l'échelle)

💡 Exemple : Black Friday sur Amazon

Pendant le Black Friday, les services **Produits** et **Commandes** sont très sollicités.
Solution : Multiplier uniquement les instances de ces services (**Scale UP**).



4.4 Inconvénients des Microservices

⚠ Complexité d'un système distribué

1. Communication complexe

- Gestion des appels réseau entre services
- Latence des appels distants

2. Plus de ressources nécessaires

- Plusieurs bases de données à gérer
- Infrastructure plus complexe

3. Tests et débogage difficiles

- Tests d'intégration complexes
- Logs distribués sur plusieurs services

4.5 Qui utilise les Microservices ?

NETFLIX Amazon Uber

SoundCloud eBay

Ces entreprises gèrent des centaines de microservices.

5 Communication entre Services

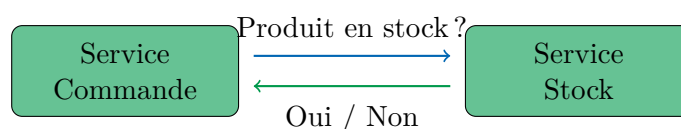
5.1 Les trois modes de communication

microblue !80	
Synchrone	Le service appelant attend une réponse
Asynchrone point à point	Le service envoie un message et continue
Diffusion d'événements	Un service notifie plusieurs services

5.2 Communication Synchrone

Appel Synchrone

Un service appelle un autre service et **attend une réponse** avant de continuer.

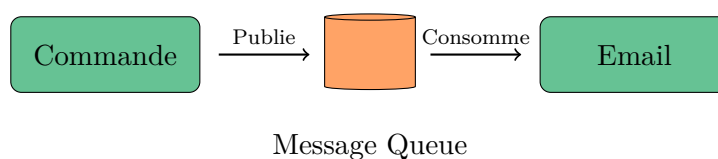


Protocoles : HTTP/REST, gRPC

5.3 Communication Asynchrone

Appel Asynchrone (Fire and Forget)

Un service envoie un message et **continue sans attendre** de réponse.

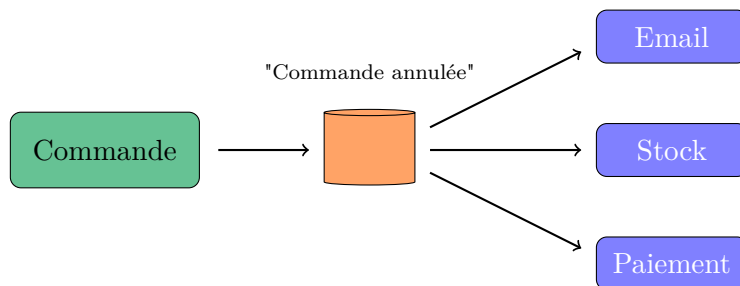


Technologies : RabbitMQ, Apache Kafka, Amazon SQS

5.4 Diffusion d'Événements

Diffusion d'Événements

Un service publie un événement. Tous les services **abonnés** le reçoivent.



5.5 Comparaison des modes

microblue !80			
Attend réponse	Oui	Non	Non
Couplage	Fort	Moyen	Faible
Exemple	Vérif. stock	Envoi email	Notifications

6 API Gateway

6.1 Définition

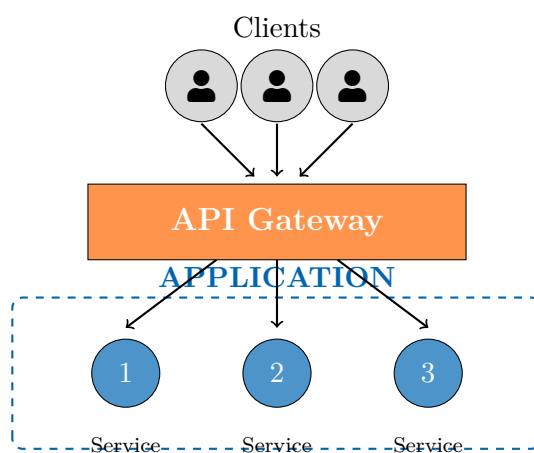
API Gateway

L'**API Gateway** est le **point d'entrée unique** entre les clients et les microservices.

Responsabilités :

- **Routing** : diriger les requêtes vers le bon service
- **Authentification** : vérifier les tokens JWT
- **Load Balancing** : répartir la charge
- **Rate Limiting** : limiter les requêtes

6.2 Architecture avec API Gateway



6.3 Technologies

microblue !80	
Kong	Open source, basé sur Nginx
AWS API Gateway	Service managé Amazon
Nginx	Serveur web configurable
Express Gateway	Basé sur Node.js

7 Étude de Cas : Gestion de Bibliothèque

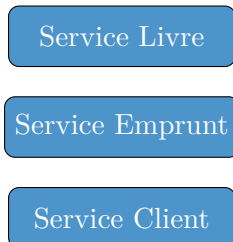
7.1 Contexte

On souhaite créer une architecture microservices pour une **bibliothèque** :

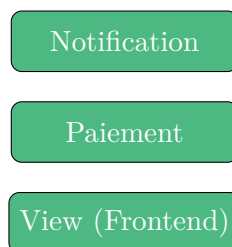
- Gestion des livres (CRUD)
- Gestion des clients
- Gestion des emprunts
- Notifications
- Paiement des frais

7.2 Identification des services

Services Métier



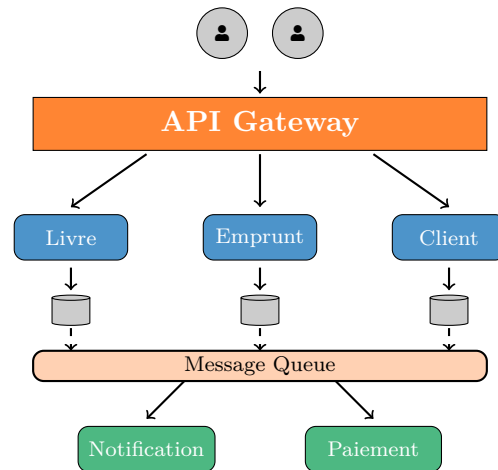
Services Utilitaires



7.3 API du Service Livre

microblue !80		
Obtenir un livre	GET /api/livres/ :id	200, 404
Ajouter un livre	POST /api/livres	201, 400
Modifier un livre	PUT /api/livres/ :id	200, 404
Supprimer un livre	DELETE /api/livres/ :id	200, 404

7.4 Architecture complète



8 Synthèse

Points clés à retenir

1. **Monolithique** = une seule unité de déploiement
2. **Microservices** = plusieurs services indépendants
3. Chaque microservice a sa **propre base de données**
4. Communication via **REST** ou **Message Queues**
5. **API Gateway** = point d'entrée unique

8.1 Comparatif

microblue !80		
Déploiement	Tout ensemble	Indépendant
Scalabilité	Globale	Ciblée
Technologies	Une seule	Multiples
Base de données	Partagée	Par service
Complexité	Simple	Plus complexe

8.2 Quand choisir quoi ?

Monolithique

- ✓ Petite équipe
- ✓ MVP / Prototype
- ✓ Déploiement rare

Microservices

- ✓ Grande équipe
- ✓ Scalabilité requise
- ✓ Déploiement fréquent